Algoritmos de optimización - Trabajo Práctico

Nombre y Apellidos: Carlos Javier Bravo Intriago

Url: https://github.com/carlosbravo1408/03MIAR-Algoritmos-de-Optimizacion-2025/tree/TrabajoPractico/TrabajoPractico

Google Colab: https://colab.research.google.com/drive/161T3DH8s50KI7pa2-tEvWEOha7hKmSWt?usp=sharing

Problema:

> 3. Configuración de Tribunales

Descripción del problema:

- Se precisa configurar tribunales de evaluación para un grupo de 15 alumnos que desean presentar su Trabajo Fin de Máster (TFM).
- Cada tribunal está compuesto por tres profesores, cada uno desempeñando uno de los siguientes roles: Presidente, Secretario o Vocal.
- Los profesores han indicado su disponibilidad horaria para participar en los tribunales de 15h a 21h durante la semana del 15 al 19 de abril:

> Número de profesores : 10
> Número de tribunales : 15
> Disponibilidad/Roles : https://bit.ly/41QWk8o
>
> - 1 indica que profesor tiene disponibilidad
> - 0 en caso contrario

#	Dia	15							16							17							18							19						
	Profesor/Hora	15	16	17	18	19	20	21	15	16	17	18	19	20	21	15	16	17	18	19	20	21	15	16	17	18	19	20	21	15	16	17	18	19	20	21
1	RRD	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	0	0
2	QYV	1	1	1	1	0	0	0	0	1	1	1	1	0	0	1	0	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	LHL	0	0	1	1	0	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	0	1	1	1	0	1	0	1	1	0	1	0	1
4	HLC	1	0	1	0	1	1	0	1	0	0	1	1	1	1	0	0	1	1	1	1	1	1	0	1	1	0	1	1	1	1	1	1	1	1	0
5	MSB	1	1	0	1	0	1	1	1	1	1	0	1	1	1	1	0	1	1	0	1	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0
6	PMQ	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	0	1	0	1
7	QWF	0	1	1	1	1	1	1	1	1	0	1	1	0	1	0	0	1	1	0	0	1	1	0	0	0	0	1	1	1	1	1	1	1	0	1
8	EBB	1	1	1	1	1	0	0	1	1	0	1	1	1	0	1	1	1	0	0	1	1	0	1	1	1	1	1	1	0	1	1	1	0	1	0
9	IOE	1	0	1	1	0	1	0	0	1	1	1	1	1	1	1	1	0	0	0	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1	1
10	IOA	1	1	0	1	1	0	1	1	0	0	0	0	0	1	1	1	0	0	1	1	1	1	0	0	1	1	1	1	1	1	1	0	0	0	1

Figura 1. Disponibilidad de docentes por fecha y hora.

Profesor	ROL
RRD	P,S,V
QYV	P,S,V
LHL	P,V
HLC	S,V
MSB	P,S,V
PMQ	P,S,V
QWF	S,V
EBB	S,V
IOE	P,S,V
IOA	P,S,V

Figura 2. Roles de cada docente.

- Hay 15 alumnos, por lo que se deben configurar 15 tribunales buscando la configuración más equilibrada posible en cuanto a la cantidad de tribunales asignados a cada profesor, es decir, evitando que un profesor tenga muchos tribunales y otros pocos.
- Obviamente, ningún profesor puede asistir a dos tribunales a la misma fecha/hora y no puede ser convocado a un tribunal al que no tiene disponibilidad.

Transformaciones de datos previa:

Se propone aplicar las siguientes transformaciones estructurales a los datos de entrada:

1. Codificación de Roles mediante máscaras de Bits:

Para evaluar la compatibilidad de los 3 roles exigidos en un tribunal, se descarta el uso de cadenas de texto o arreglos de atributos. En su lugar, se propone usar una notación binaria de 3 bits. Cada posición del bit representa la habilitación de un profesor para un rol específico:

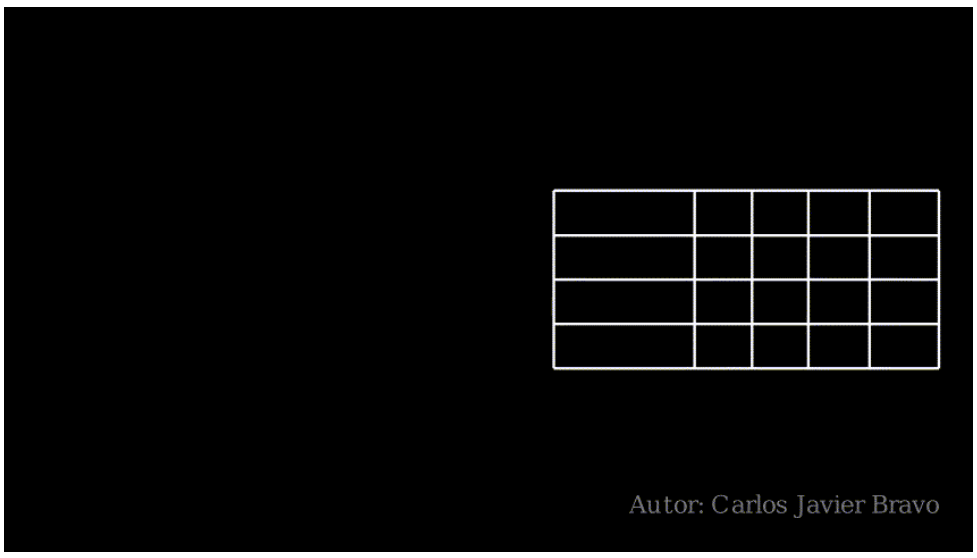
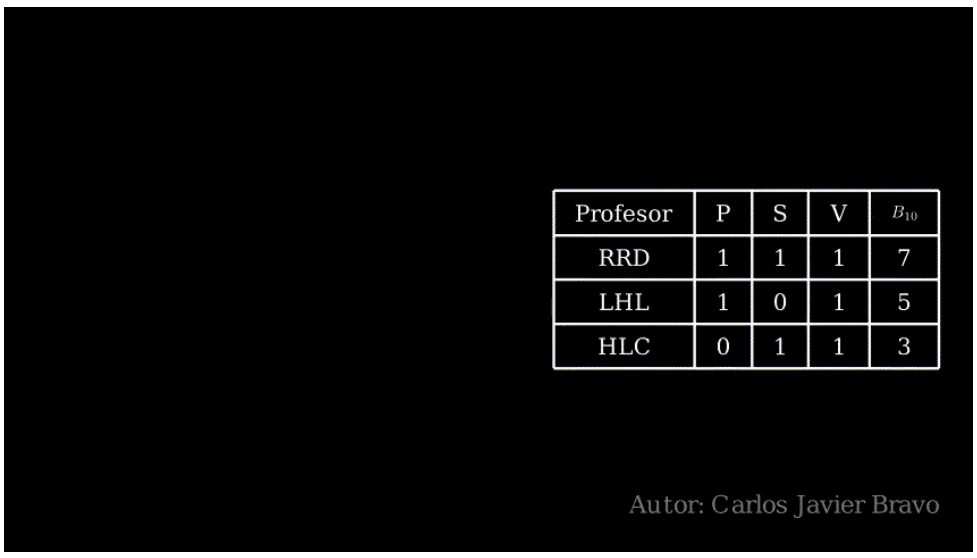


Figura 3. Codificación de Roles mediante máscaras de Bits.

De esta forma, un profesor que puede ser Presidente, Secretario y Vocal tendrá el valor $7\ (4+2+1)$. La verificación de sí un profesor cumple con un rol requerido se reduce a una única operación lógica AND a nivel de bits entre el rol del profesor y la máscara del rol que necesitas, lo cual se ejecuta en un solo ciclo de reloj del procesador.



Profesor	P	S	V	B_{10}
RRD	1	1	1	7
LHL	1	0	1	5
HLC	0	1	1	3

Figura 4. Validacion de cumplimiento de rol.

2. Profesores:

Para aislar la lógica del algoritmo de los metadatos de los profesores, se sustituyen las cadenas de texto (nombres o identificaciones reales) por identificadores numéricos primitivos (enteros consecutivos). Se asigna a cada recurso docente un ID único partiendo desde cero.

```
NOMBRE_PROFESORES = {
    0: "RRD", 1: "QYV", 2: "LHL", 3: "HLC", 4: "MSB",
    5: "PMQ", 6: "QWF", 7: "EBB", 8: "IOE", 9: "IOA"
}
```

3. Disponibilidad

Se implementa como un diccionario indexado que apunta a arreglos anidados.

- Clave de acceso (Key): El identificador numérico único del profesor $(ID \in [0, 9])$.
- Valor de retorno (Value): Una matriz discreta de 5×7 que modela el bloque temporal del evento. Las filas de esta matriz (índice externo) representan los días de la semana (de lunes a viernes), mientras que las columnas (índice interno) representan las franjas horarias disponibles.

In [1]:
```python
import copy
import os
import math
import random
from typing import List, Set, Tuple, Union, Optional, Literal, FrozenSet
import pickle
import matplotlib.pyplot as plt
from collections import Counter, defaultdict

import pandas as pd
import numpy as np

NOMBRE_PROFESORES = {
    0: "RRD",
    1: "QYV",
    2: "LHL",
    3: "HLC",
    4: "MSB",
    5: "PMQ",
    6: "QWF",
    7: "EBB",
    8: "IOE",
    9: "IOA"
}
```

```
ROLES = {
#  id: P   S   V
    0: 4 | 2 | 1,  # B₂=111, B₁₀=7
    1: 4 | 2 | 1,  # B₂=111, B₁₀=7
    2: 4 | 1,      # B₂=101, B₁₀=5
    3: 2 | 1,      # B₂=011, B₁₀=3
    4: 4 | 2 | 1,  # B₂=111, B₁₀=7
    5: 4 | 2 | 1,  # B₂=111, B₁₀=7
    6: 2 | 1,      # B₂=011, B₁₀=3
    7: 2 | 1,      # B₂=011, B₁₀=3
    8: 4 | 2 | 1,  # B₂=111, B₁₀=7
    9: 4 | 2 | 1   # B₂=111, B₁₀=7
}
DISPONIBILIDAD = {
    #                   d0                          d1                              dn
    # ID_P:[[h0,h1,h2,h3,h4,h5,h6,h7], [h0,h1,h2,h3,h4,h5,h6,h7], ..., [h0,h1,h2,h3,h4,h5,h6,h7]]
    0: [[0,1,1,1,0,1,1], [1,0,1,1,1,1,1], [1,1,0,0,1,0,1], [0,1,1,1,1,1,1], [1,1,1,1,1,0,0]],
    1: [[1,1,1,1,0,0,0], [0,1,1,1,1,0,0], [1,0,0,1,1,1,0], [1,1,1,1,1,1,1], [1,1,1,1,1,1,1]],
    2: [[0,0,1,1,0,1,1], [1,1,1,0,0,1,1], [1,1,1,1,1,1,1], [1,0,1,1,1,0,1], [0,1,1,0,1,0,1]],
    3: [[1,0,1,0,1,1,0], [1,0,0,1,1,1,1], [0,0,1,1,1,1,1], [1,0,1,1,0,1,1], [1,1,1,1,1,1,0]],
    4: [[1,1,0,1,0,1,1], [1,1,1,0,1,1,1], [1,0,1,1,0,1,1], [0,1,1,1,0,1,1], [1,0,1,1,1,1,0]],
    5: [[1,1,1,1,1,0,0], [1,1,1,1,1,1,1], [1,1,0,0,1,1,1], [1,1,1,0,0,1,1], [1,1,1,0,1,0,1]],
    6: [[0,1,1,1,1,1,1], [1,1,0,1,1,0,1], [0,0,1,1,0,0,1], [1,0,0,0,0,1,1], [1,1,1,1,1,0,1]],
    7: [[1,1,1,1,1,0,0], [1,1,0,1,1,1,0], [1,1,1,0,0,1,1], [0,1,1,1,1,1,1], [0,1,1,1,0,1,0]],
    8: [[1,0,1,1,0,1,0], [0,1,1,1,1,1,1], [1,1,0,0,0,1,1], [1,1,1,1,1,1,0], [1,0,1,1,1,1,1]],
    9: [[1,1,0,1,1,0,1], [1,0,0,0,0,0,1], [1,1,0,0,1,1,1], [1,0,0,1,1,1,1], [1,1,1,0,0,0,1]],
}

SolutionType = List[List[int]]
```

Modelo

¿Cómo representó el espacio de soluciones?

Para modelar el espacio de soluciones de este problema de asignación de horarios acotado a repartir dichos horarios de forma que no existan profesores con más carga laboral que otros. La estructura de datos (después de una serie de pruebas y errores) descarta el enfoque de indexación por bloques de tiempo (con la idea de permitir N defensas de Trabajo Final de Titulación (TFT) en un mismo horario de ser admisible). En su lugar adopta un enfoque centrado en las defensas de TFT.

La solución se representa como una matriz bidimensional homogénea de enteros de tamaño fijo 15×5. Cada fila de la matriz corresponde a un TFT específico.

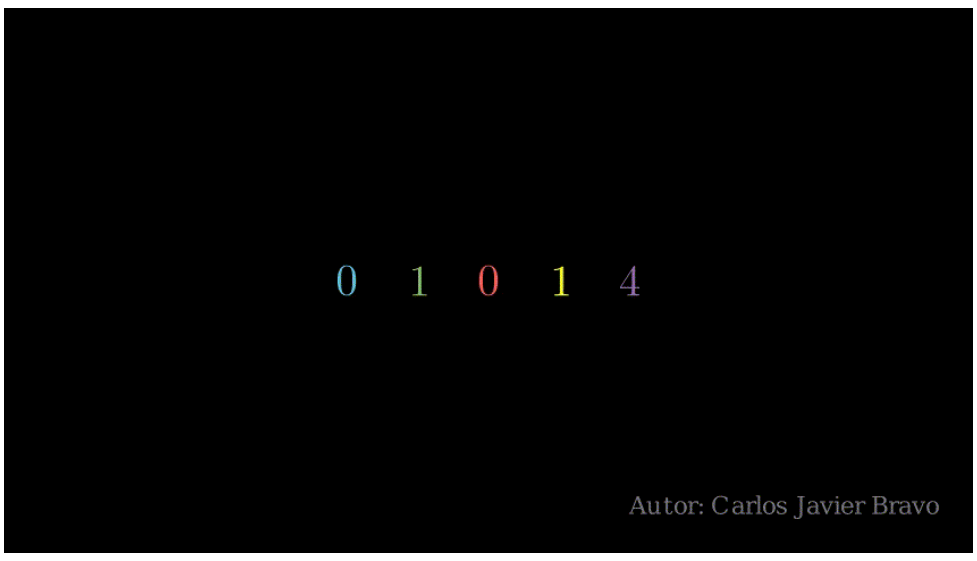


Figura 5. Representacion de la Solución.

La solución S se define como un conjunto de vectores S_i para $0 \leq i \leq 14$, donde cada vector tiene la forma:

$$S_i = [d, h, p, s, v]$$

Los componentes del vector se codifican mediante identificadores numéricos enteros.

- $d \in 0, 1, 2, 3, 4$: Representa el dia de la semana (del 15 al 19 de abril).
- $h \in 0, 1, 2, 3, 4, 5, 6$: Representa el bloque horario (de 15h00 a 21h00)
- $p, s, v \in 0, 1, \ldots, 9$: representa los IDs únicos de los profesores asignados a los roles de Presidente, Secretario y Vocal respectivamente.

Justificación del diseño

- Permite que multiples vectores S_i contengan la misma fecha y el mismo bloque horario (d, h). Esto habilita la programacion de hasta $\lfloor 10/3 \rfloor$ tribunales paralelos en la misma franja horaria.
- Facilita la aplicación de operadores de perturbación tales como cambios de profesores, reubicaciones temporales, etc.

¿Cuál es la función objetivo?

El problema requiere satisfacer reglas que no se pueden violar (penalidades duras), mientras se optimiza la distribución *equitativa* del trabajo para los docentes (costos suaves).

Dicho esto, el sistema se evalúa mediante una función de coste $F(S)$ que el algoritmo buscara minimizar:

$$F(S) = W_{hard} \cdot P_{hard} + W_{soft} \cdot P_{soft}$$

Medición de la *equidad* laboral o costo suave P_{soft}

Para evaluar el balance de carga de los docentes considerando que el sistema requiere cubrir 45 posiciones (15 tribunales $\times$ 3 roles) utilizando 10 profesores, lo que establece una media de $\mu = 4.5$ participaciones por profesor.

El balance (o penalización suave) se calcula a partir de la suma de las diferencias al cuadrado. Sea x_i el número total de participaciones asignadas al profesor i:

$$P_{soft} = \sum_{i=0}^{9} (x_i - \mu)^2$$

Note que tiene parecido a la ecuación de la Varianza, sin embargo, se ha omitido el coeficiente $\frac{1}{10}$ para simplificar el peso W_{soft}. Con esta omisión el valor de W_{soft} es 1, en cambio, si se usa la ecuación completa de la varianza, el valor de W_{soft} es 10.

Componente de Penalización Dura (P_{hard}):

La idea de este componente es cuantificar las violaciones a las reglas físicas y lógicas del problema. Representa la suma total de tres tipos de fallos:

1. Indisponibilidad de profesores: Cuantos profesores no están disponibles en ese bloque horario.
2. Solapamiento: Número de asignaciones donde un mismo profesor es requerido en dos o más tribunales paralelos en el mismo bloque horario.
3. Incompatibilidad con el rol: Número de profesores a los cuales se les asignó un rol para el cual no están habilitados.

Calibración del peso W_{hard} basándose en la estimación del peor escenario para P_{soft}:

Dado que se requiere garantizar que el algoritmo no acepte una solución inválida (dado alguno de los tres tipos de fallos anteriormente mencionados) y manteniendo un equilibro de carga laboral, el peso W_{hard} se establece mediante el cálculo de la cota superior del peor escenario posible para P_{soft}.

El desequilibrio máximo ocurre si 3 profesores acaparan o asumen todos los tribunales cada uno $(x_i = 15)$, y los 7 restantes no asumen ningún tribunal $(x_i = 0)$. Evaluando esto en función de la suma de cuadrados:

$$P_{max} = 3 \cdot (15 - 4.5)^2 + 7 \cdot (0 - 4.5)^2 = 330.75 + 141.75 = 472.5$$

Dado que el peor coste *suave* posible es de 472.5, y estableciendo $W_{soft} = 1$, se asigna un valor *seguro* de $W_{hard} \geq 472.5$.

In [2]:
```
TFT_N = 15
mean = 3 * TFT_N / len(NOMBRE_PROFESORES)
W_hard = 3 * (15 - mean) ** 2 + 7 * mean ** 2 # 472.5
W_soft = 1

def costo_total(solucion: SolutionType) -> float:
    participaciones = [0 for _ in range(len(NOMBRE_PROFESORES))]

    for slot in solucion:
        for i in range(2, 5):
            participaciones[slot[i]] += 1

    p_soft = sum([(c - mean)**2 for c in participaciones])
    p_hard = 0
    p_hard += penalizacion_por_indisponibilidad(solucion)
    p_hard += penalizacion_por_solapamiento(solucion)
    p_hard += penalizacion_por_rol(solucion)

    return p_hard * W_hard + p_soft * W_soft
```

¿Cómo implemento las restricciones?

Las restricciones operativas del sistema (disponibilidad, no solapamiento y compatibilidad de roles) se implementan como funciones de penalización independientes (Hard Constraints). En lugar de aplicar una poda estricta que descarte soluciones inválidas, cada función evalúa la matriz de solución S (de 15×5) y retorna un número entero que representa la cantidad exacta de infracciones. Esto proporciona un gradiente numérico continuo que guía a la metaheurística hacia la factibilidad.

Restricción de Disponibilidad horaria:

Procura garantizar que ningún profesor sea convocado en un horario bloqueado. Se itera sobre las 15 filas de la matriz S. Para cada fila se extrae las coordenadas de tiempo, y se evalúa a los tres profesores asignados su disponibilidad según la matriz `DISPONIBILIDAD`. Por cada consulta en la que el profesor no está disponible, se suma una unidad al contador de penalización.

Restricción de Solapamiento:

Se asegura de que, en caso de existir multiples tribunales programados para la misma fecha y hora, un profesor no sea asignado a más de un tribunal simultáneamente. Por cada solapamiento, se suma una unidad al contador de penalización.

Restricción de Roles

Se valida que el profesor asignado a una posición específica cuente con el rol necesario. De no cumplir esta restricción, se suma una unidad al contador de penalización.

In [3]:
```
def penalizacion_por_indisponibilidad(solucion: SolutionType) -> int:
    contador = 0
    for slot in solucion:
        for i in range(2, 5):
            if DISPONIBILIDAD[slot[i]][slot[0]][slot[1]] == 0:
                contador += 1
    return contador

def penalizacion_por_solapamiento(solucion: SolutionType) -> int:
    contador = 0
    defensas = defaultdict(list)
    for slot in solucion:
        defensas[(slot[0], slot[1])].append(set(slot[2:]))
    for tribunales in defensas.values():
        if len(tribunales) <= 1:
            continue
        cantidad_esperada = len(tribunales) * 3
        profesores = set()
        for tribunal in tribunales:
            profesores.update(tribunal)
        contador += cantidad_esperada - len(profesores)
    return contador

def penalizacion_por_rol(solucion: SolutionType) -> int:
    contador = 0
    for slot in solucion:
        for i in range(2, 5):
            rol = 4 if i == 2 else 2 if i == 3 else 1
            contador += ((rol & ROLES[slot[i]]) == 0)
    return contador
```

Análisis

¿Qué complejidad tiene el problema? Orden de complejidad y Contabilizar el espacio de soluciones

Complejidad del problema y Orden de complejidad:

Este problema pertenece a la clase de complejidad **NP-Hard**, lo que significa que **no** hay un algoritmo conocido que pueda resolverlo en tiempo polinomial. Pero si se puede verificar cualquier solución en un tiempo polinómico, en este caso simplemente se suma los actores y se revisa la restricción de 6 tomas máximas por día.

Si se tratase de resolver por *fuerza bruta*, la complejidad para este problema es exponencial.

Contabilización del espacio de Soluciones:

Para calcular el número total de soluciones posibles (tanto las factibles como las no factibles) que el algoritmo puede explorar, se debe desglosar la combinatoria de un unico TFT, y luego escalarlo a los 15 TFTs.

Para un unico TFT, la asignación se divide en dos dimensiones:

- La dimension temporal, el calendario cuenta con 5 días y 7 franjas horarias por día, dando 35 opciones temporales.
- La dimensio de recursos, Se requiere elegir a 3 profesores distintos de una plantilla de 10, donde el orden de selección es estrictamente importante debido a los roles. Esto obedece a una permutación sin repetición:

$$P(10, 3) = 720 \text{combinaciones del tribunal}$$

- Multiplicando ambas dimensiones, un solo TFT puede configurarse de $35 \times 720 = 25200$ formas distintas.

Dado que el sistema exige programar 15 TFTs de manera independiente dentro de la matriz solución, el espacio de soluciones totales crece exponencialmente:

$$\text{Espacio Total} = 25200^{15} = 1.0496 \times 10^{66}$$

El espacio de soluciones factibles es menor, y un algoritmo *determinista* probablemente navegue por todo ese espacio total. Debido a esto se hará uso de estrategias heurísticas para buscar llegar al mínimo global, sin tener que explorar todo el espacio de solución teórico.

Diseño

¿Qué técnica utilizo? ¿Por qué?

Se implementó la metaheurística de la trayectoria estocástica de Recocido Simulado (SA).

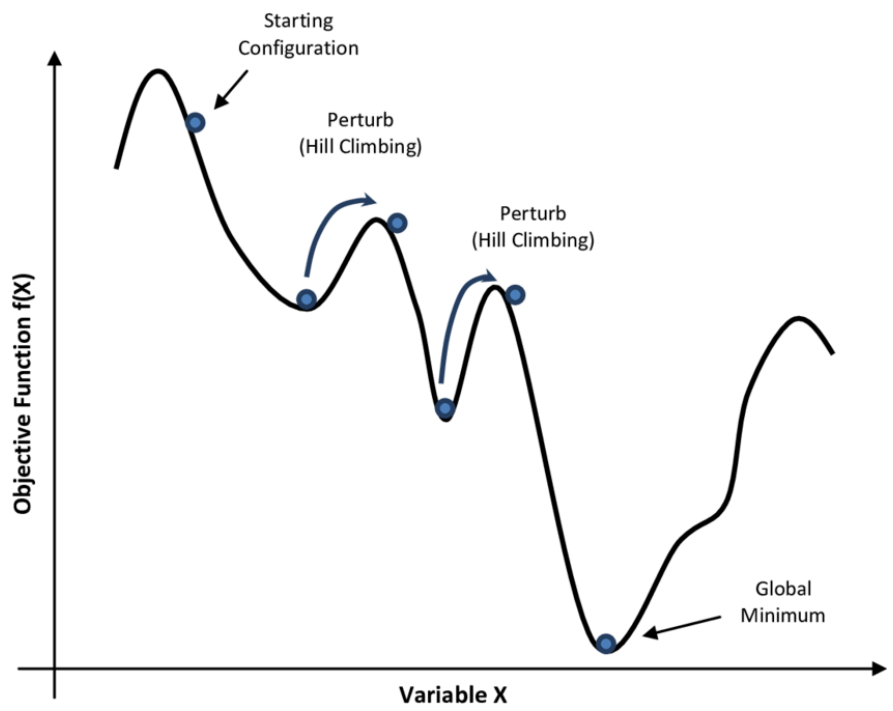


Fig 6. Representación del algoritmo SA. *Tomado de [2]*

La selección de SA se debe a tres *criterios* críticos impuestos por la naturaleza del problema (restricciones).

La penalización alta de las restricciones duras $(W_{hard} \geq 472.5)$ tiende a generar un paisaje de búsqueda extremadamente rugoso. Un algoritmo de búsqueda local, se congelaría en el primer minimo local que encuentre. Para poder salir de aquellos mínimos locales se plantea cuatro tipos de perturbaciones con la finalidad de llegar a instancias donde $P_{hard} = 0$.

Generación de nueva vecindad:

La naturaleza del problema altamente rugoso requiere una estrategia que se plantea en dos fases distintas:

1. Fase de Factibilidad: Debido a que al inicio y por la topología del problema, el algoritmo necesita realizar saltos más amplios, bruscos (exploración destructiva) para encontrar el valle de soluciones válidas.
2. Fase de Optimización: una vez que $P_{hard} = 0$, el algoritmo necesita realizar ajustes finos (una exploración conservadora) para equilibrar la varianza de la carga de los profesores $P_{soft} = 0$ sin romper las reglas duras que ya se cumplieron.

Fase de Factibilidad:

1. Reubicación temporal:

Este movimiento apunta a arreglar profesores indisponibles, corrige roles erróneos y altera directamente el balance de la varianza. Debe ser el movimiento más frecuente.

- Mecanismo: Selecciona una fila (TFT) al azar entre las 15 disponibles. Genera un nuevo Día (0-4) y una nueva Hora (0-6) al azar. Sustituye los índices 0 y 1 de esa fila.
- Impacto: Deja intacta la asignación de profesores y su varianza, pero mueve el bloque entero a otra coordenada del calendario.

2. Reemplazo de Recursos (Profesores):

Mecanismo para resolver solapamientos masivos. Mover un bloque entero de tiempo permite escapar de óptimos locales donde hay demasiados tribunales agrupados en el mismo día

- Mecanismo: Selecciona una fila al azar. Selecciona una columna de rol al azar (índices 2, 3 o 4). Sustituye el ID del profesor actual por un nuevo ID aleatorio entre 0 y 9.

Fase de Optimización:

3. Intercambio externo:

Este movimiento apunta a explorar el espacio de varianza (P_{soft}) sin alterar la topología de tiempo. Es un movimiento conservador. Solo sirve para pulir el P_{soft} intercambiando cargas de trabajo entre dos tribunales ya existentes.

- Mecanismo: Selecciona dos filas distintas al azar (TFM A y TFM B). Selecciona una columna de rol al azar (ej. índice 2, Presidente). Intercambia el Presidente del TFM A con el Presidente del TFM B.
- Impacto: Como ambos profesores asumen el mismo rol que el otro dejó, la suma total de participaciones en el sistema no cambia, pero sus varianzas individuales sí se ven afectadas.

4. Intercambio interno:

Este movimiento apunta a corregir penalizaciones de rol exclusivamente. Es un movimiento de micro-ajuste. Solo sirve para arreglar errores de rol muy específicos sin alterar la topología temporal ni la varianza de los profesores implicados

- Mecanismo: Selecciona una fila al azar. Selecciona dos columnas de rol distintas dentro de esa fila (ej. Presidente y Vocal). Intercambia sus IDs de profesor.
- Impacto: No altera el horario ni la carga de trabajo total de esos profesores, pero puede resolver un error donde un profesor con rol "solo Vocal" estaba asignado erróneamente como Presidente.

Se propone asignar una probabilidad relativamente alta a aquellos movimientos que cumplan la Fase de Factibilidad (Reubicación Temporal y Reemplazo de Recursos), a ambas se les asignara un 80% de probabilidad, las cuales cada uno de manera equiprobable se asigna un 40% de probabilidad. Con esto se pretende destruir las soluciones inválidas rápidamente.

Los movimientos más suaves o no destructivos se les asigna el 10% a cada uno respectivamente.

Movimiento	Probabilidad
Reubicación temporal	40%
Reemplazo de Recursos	40%
Intercambio externo	10%
Intercambio interno	10%

Esquema de Enfriamiento:

Para el control de la temperatura (T), se ha descartado el enfriamiento geométrico tradicional, haciendo uso de la ecuación de Lundy-Mees [2] $\left(T_{k+1} = \frac{T_k}{1+\beta \cdot T_K}\right)$. Esta fórmula provoca un descenso rápido cuando T es alta (evitando el desperdicio de cómputo en búsquedas aleatorias excesivas) y, crucialmente, ralentiza el enfriamiento de forma extrema cuando T se acerca a cero. Esto permite una fase de "recocido lento" o fine-tuning mucho más exhaustiva en las iteraciones finales, aumentando la probabilidad de encontrar el óptimo global en instancias complejas del TSP.

Estimación del minimo global:

Para estimar cuál sería el minimo global teórico, asumimos el escenario donde la equidad es maxima (suma de cuadrados cercana a cero) y $P_{hard} = 0$.

Para ello partimos de la media $\mu = 4.5$, se conoce que no existen *medios tribunales*, por lo que el equilibrio perfecto exige que la mitad de los profesores asistan a 5 tribunales, y la otra mitad asistan a 4.

Si se aplica la fórmula de la suma de cuadrados a este escenario ideal:

$$P_{soft} = 5 \cdot (4 - 4.5)^2 + 5 \cdot (5 - 4.5)^2 = 1.25 + 1.25 = 2.5$$

In [4]:
```
def probabilidad(T: float, d: float) -> bool:
    """
    Función de probabilidad para aceptar peores soluciones
    :param T: Temperatura
    :param d:
    :return:
    """
    if d < 0: return True
    if T <= 1e-9: return False
    return random.random() < math.exp(-d / T)

def bajar_temperatura_lundy_mees(T: float, beta: float = 0.0001) -> float:
    """
    Función de descenso de temperatura basado en la ecuación de Lundy-Mess
    :param T: Temperatura
    :param beta: coeficiente
    :return:
    """
    return T / (1 + beta * T)

def reubicacion_temporal(solucion: SolutionType) -> Tuple[SolutionType, float]:
    _solucion = copy.deepcopy(solucion)
    tft_random = random.randint(0, len(solucion) - 1)
```

```python
    dia_random = random.randint(0, 4)
    hora_random = random.randint(0, 6)
    _solucion[tft_random][0] = dia_random
    _solucion[tft_random][1] = hora_random
    return _solucion, costo_total(_solucion)

def reemplazo_de_recurso(solucion: SolutionType) -> Tuple[SolutionType, float]:
    _solucion = copy.deepcopy(solucion)
    tft_random = random.randint(0, len(solucion) - 1)
    while True:
        rol_random = random.randint(2, 4)
        profesor_random = random.randint(0, 9)
        if profesor_random not in _solucion[tft_random][2:]:
            _solucion[tft_random][rol_random] = profesor_random
            break
    return _solucion, costo_total(_solucion)

def intercambio_externo(solucion: SolutionType) -> Tuple[SolutionType, float]:
    _solucion = list(solucion)
    while True:
        idx1, idx2 = random.sample(range(len(_solucion)), 2)
        rol_random = random.randint(2, 4)
        prof1 = _solucion[idx1][rol_random]
        prof2 = _solucion[idx2][rol_random]
        if prof1 not in _solucion[idx2][2:] and prof2 not in _solucion[idx1][2:]:
            _solucion[idx1] = list(_solucion[idx1])
            _solucion[idx2] = list(_solucion[idx2])

            _solucion[idx1][rol_random] = prof2
            _solucion[idx2][rol_random] = prof1
            break
    return _solucion, costo_total(_solucion)

def intercambio_interno(solucion: SolutionType) -> Tuple[SolutionType, float]:
    _solucion = copy.deepcopy(solucion)
    tft_random = random.randint(0, len(solucion) - 1)
    roles_random = random.sample(range(2, 5), 2)
    aux = _solucion[tft_random][roles_random[0]]
    _solucion[tft_random][roles_random[0]] = _solucion[tft_random][roles_random[1]]
    _solucion[tft_random][roles_random[1]] = aux
    return _solucion, costo_total(_solucion)

def generar_vecina_sa(solucion: SolutionType) -> Tuple[SolutionType, float]:
    prob = random.random()
    if prob < 0.4:
        return reubicacion_temporal(solucion)
    if prob < 0.8:
        return reemplazo_de_recurso(solucion)
    if prob < 0.9:
        return intercambio_externo(solucion)
    return intercambio_interno(solucion)

def generar_solucion_aleatoria() -> SolutionType:
    solucion = []
    for _ in range(15):
        dia = random.randint(0, 4)
        hora = random.randint(0, 6)
        profesores = random.sample(range(10), 3)
        solucion.append([dia, hora, profesores[0], profesores[1], profesores[2]])
    return solucion
```

In [5]:

```python
def recocido_simulado(
        TEMPERATURA: float,
        solucion_inicial: Optional[SolutionType] = None,
        min_temp: float = 0.0001,
        verbose: bool = False
) -> Tuple[SolutionType, float]:
    if solucion_inicial:
        solucion_referencia = copy.deepcopy(solucion_inicial)
    else:
        solucion_referencia = generar_solucion_aleatoria()
    distancia_referencia = costo_total(solucion_referencia)

    mejor_solucion = copy.deepcopy(solucion_referencia)  #x* del pseudocodigo
    mejor_distancia = distancia_referencia  #F* del pseudocodigo

    N = 0
    last_n = 0

    while TEMPERATURA > min_temp: # no converge
        N += 1
        #Genera una solución vecina
        vecina, costo_vecina = generar_vecina_sa(solucion_referencia)
        delta = costo_vecina - distancia_referencia
        #Si la nueva vecina es mejor se cambia
        #Si es peor se cambia según una probabilidad que depende de T y delta
        # (distancia_referencia - distancia_vecina)
```

```
        if delta < 0 or probabilidad(TEMPERATURA, delta):
            solucion_referencia = copy.deepcopy(vecina)
            distancia_referencia = costo_vecina

            #Si es la mejor solución de todas se guarda(siempre!!!)
            if distancia_referencia < mejor_distancia:
                last_n = N
                mejor_solucion = copy.deepcopy(vecina)
                mejor_distancia = distancia_referencia

        #Bajamos la temperatura
        TEMPERATURA = bajar_temperatura_lundy_mees(TEMPERATURA)
    # Ordenamos la solucion por fechas
    mejor_solucion.sort(key=lambda x: tuple(x))
    if verbose:
        print(
            f"La mejor solución encontrada después de {last_n} iteraciones, "
            f"es ", end=""
        )
        print(mejor_solucion, mejor_distancia)
    return mejor_solucion, mejor_distancia
```

In [6]:
```
solucion_final, costo = recocido_simulado(
    TEMPERATURA=100,min_temp=0.05, verbose=True
)
```

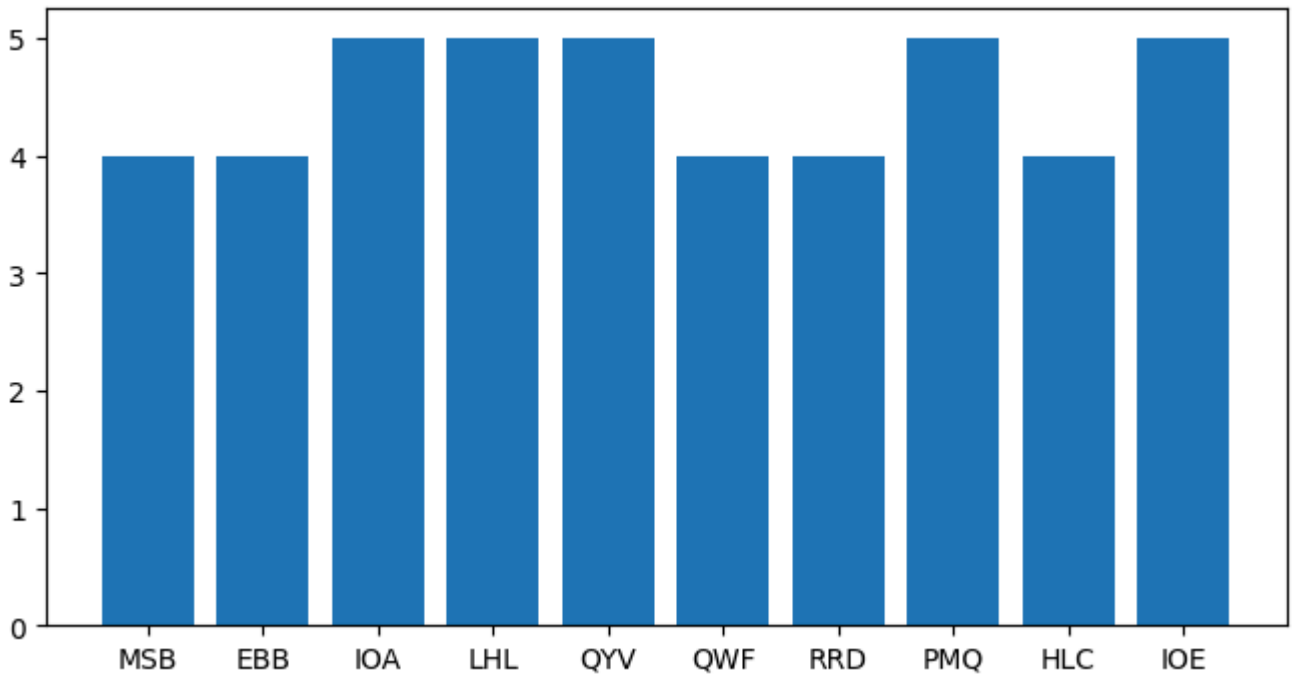
```
La mejor solución encontrada después de 4661 iteraciones, es [[0, 0, 4, 7, 9], [0, 2, 2, 1, 6], [0, 3, 4, 0, 9], [0, 4, 5, 6, 7], [1, 0, 9, 3, 4], [1, 1, 4, 6, 5], [1, 3, 8, 3, 7], [2, 0, 8, 0, 1], [2, 1, 0, 8, 2], [2, 6, 2, 8, 5], [3, 2, 5, 3, 2], [3, 5, 9, 5, 6], [4, 1, 2, 3, 1], [4, 2, 1, 7, 0], [4, 6, 8, 9, 1]] 2.5
```

In [7]:
```
def visualizar_resultado(solucion: SolutionType) -> None:
    fecha_inicio = 15
    hora_inicio = 15
    resultado = []
    lista_profesores = []
    for tft in solucion:
        fecha = tft[0] + fecha_inicio
        hora = tft[1] + hora_inicio
        presidente = NOMBRE_PROFESORES[tft[2]]
        secretario = NOMBRE_PROFESORES[tft[3]]
        vocal = NOMBRE_PROFESORES[tft[4]]
        resultado.append([fecha, hora, presidente, secretario, vocal])
        lista_profesores.extend([presidente, secretario, vocal])
    resultado.sort(key=lambda x: tuple(x))
    df = pd.DataFrame(resultado, columns=['Fecha', 'Hora', 'Presidente', 'Secretario', 'Vocal'])
    display(df)

    frecuencias = Counter(lista_profesores)
    plt.figure(figsize=(8, 4))
    plt.bar(frecuencias.keys(), frecuencias.values())
    plt.show()
```

In [8]:
```
visualizar_resultado(solucion_final)
```

	Fecha	Hora	Presidente	Secretario	Vocal
0	15	15	MSB	EBB	IOA
1	15	17	LHL	QYV	QWF
2	15	18	MSB	RRD	IOA
3	15	19	PMQ	QWF	EBB
4	16	15	IOA	HLC	MSB
5	16	16	MSB	QWF	PMQ
6	16	18	IOE	HLC	EBB
7	17	15	IOE	RRD	QYV
8	17	16	RRD	IOE	LHL
9	17	21	LHL	IOE	PMQ
10	18	17	PMQ	HLC	LHL
11	18	20	IOA	PMQ	QWF
12	19	16	LHL	HLC	QYV
13	19	17	QYV	EBB	RRD
14	19	21	IOE	IOA	QYV



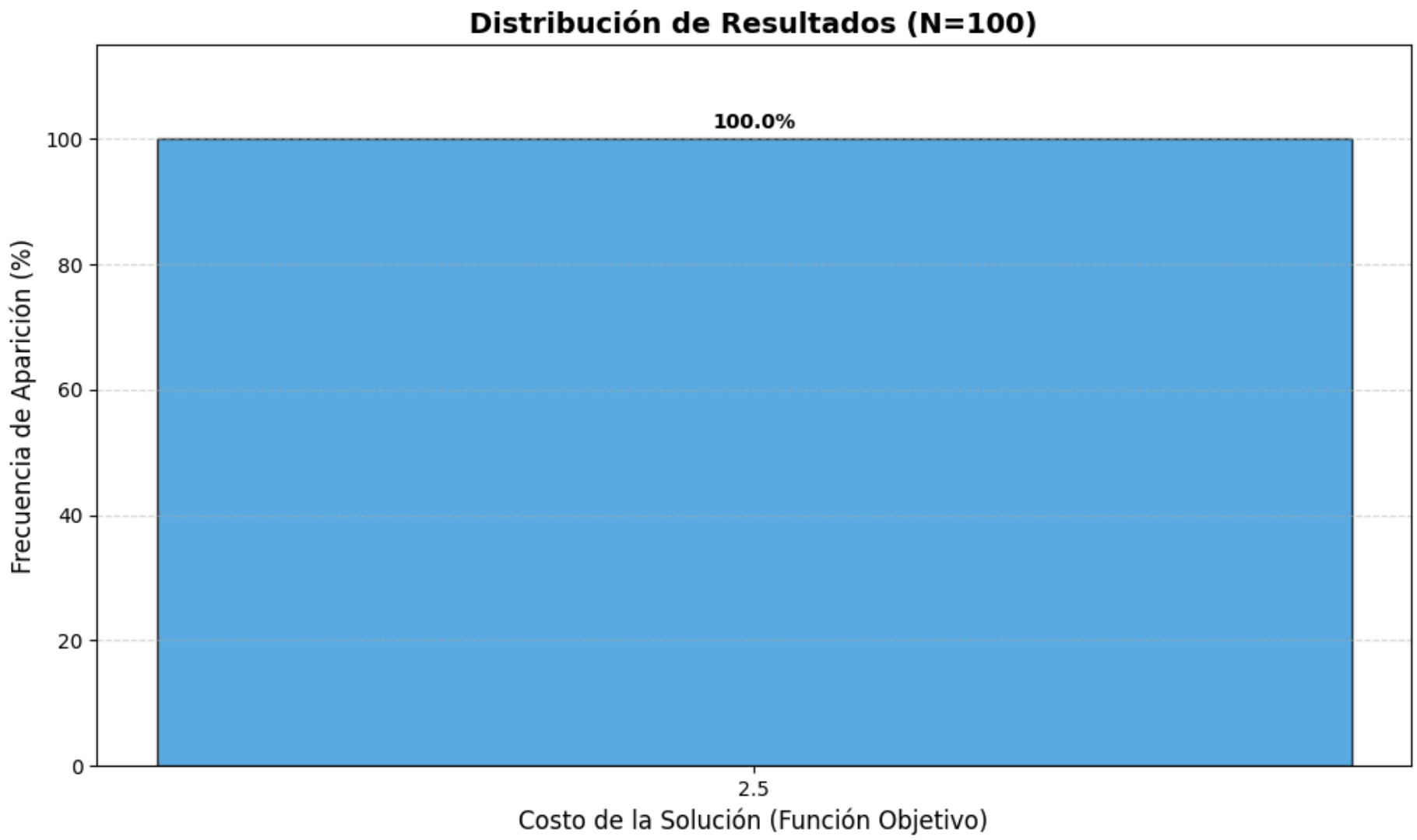
Experimento:

Se propone ejecutar 100 veces la implementación SA con los datos propuestos para determinar la frecuencia a la que el algoritmo converge al minimo global.

In [9]:
```
n = 100
costos = []
for _ in range(n):
    c_solucion, c_costo = recocido_simulado(
        TEMPERATURA=100,min_temp=0.05
    )
    costos.append(c_costo)
```

In [10]:
```
def graficar_distribucion(lista_resultados):
    """
    Genera un histograma/gráfico de barras mostrando el porcentaje
    de veces que el algoritmo encontró cada solución.\n
    Código sugerido por Gemini 3.1 Pro con modificaciones hechas por el autor
    """
    total_ejecuciones = len(lista_resultados)
    conteo = Counter(lista_resultados)
    costos_unicos = sorted(conteo.keys())
    frecuencias = [conteo[costo] for costo in costos_unicos]
    porcentajes = [(freq / total_ejecuciones) * 100 for freq in frecuencias]
    fig, ax = plt.subplots(figsize=(10, 6))
    colores = ['#3498db' for c in costos_unicos]
    barras = ax.bar(
        costos_unicos, porcentajes, color=colores, edgecolor='black', alpha=0.8
    )
    for barra in barras:
        height = barra.get_height()
        ax.annotate(
            f'{height:.1f}%',
            xy=(barra.get_x() + barra.get_width() / 2, height),
            xytext=(0, 3),
            textcoords="offset points",
            ha='center', va='bottom', fontweight='bold'
        )
    ax.set_xlabel('Costo de la Solución (Función Objetivo)', fontsize=12)
    ax.set_ylabel('Frecuencia de Aparición (%)', fontsize=12)
    ax.set_title(
        f'Distribución de Resultados '
        f'(N={total_ejecuciones})', fontsize=14, fontweight='bold'
    )
    ax.set_xticks(costos_unicos)
    ax.set_xticklabels(costos_unicos)
    ax.grid(axis='y', linestyle='--', alpha=0.5)
    plt.ylim(0, max(porcentajes) * 1.15)
    plt.tight_layout()
    plt.show()
```

In [11]:
```
graficar_distribucion(costos)
```



Bibliografia

- [1] F. Sancho Caparrini, "Simulated Annealing in NetLogo", Blog de Fernando Sancho Caparrini - Universidad de Sevilla. [En línea]. Disponible en: https://www.cs.us.es/~fsancho/Blog/posts/Simulated_Annealing_in_NetLogo.md. [Accedido: 25-feb-2026].
- [2] M. Lundy y A. Mees, "Convergence of an annealing algorithm," Mathematical Programming, vol. 34, no. 1, pp. 111–124, 1986.